MICHIGAN ENGINEERING
UNIVERSITY OF MICHIGAN

System Interfaces for Agents

Alanna Qiu, Alexander Devine, Gautham Santhosh, Mythri Srinivasa

MICHIGAN ENGINEERING
UNIVERSITY OF MICHIGAN

Agents Are Now Using Interfaces Built for Humans

	INTERFACE BUILT FOR HUMANS	WHY IT FAILS AGENTS
DMI Users of the operating system	**The GUI** Menus and buttons, operated one click at a time	Agents misread screens, and every click costs a slow model call.
CAKE Developers of GPU kernels	**The compile–test–time loop** Returns a compiler error, a test pass or fail, and a run time	Experts read those signals using knowledge of the workload and hardware. Agents lack that context.

Today's question: how can we make systems that agents can use effectively and efficiently?

MICHIGAN ENGINEERING
UNIVERSITY OF MICHIGAN

CAKE: Compiler-Agent Co-Design for Frontier Kernel Evolution

Zihao Ye*, Yingyi Huang*, Hongyi Jin, Bohan Hou, Junru Shao, Zhongming Yu, Jinqi Chen, Meghan Cowan, Shiyi Cao, Shanli Xing, Hanfeng Chen, Vinod Grover, Tianqi Chen, Luis Ceze

NVIDIA · Carnegie Mellon University · arXiv 2608.12629, August 2026 · *Equal contribution

Hand-Tuned Kernels Get More From the Same GPU

SAME ATTENTION MATH, SAME H100

35% → 67%

of peak FLOPS (≈350 → 660 TFLOPS). FlashAttention, the standard attention kernel: v2 tuned for the A100, v3 for the H100.

In sequence: tensor cores idle about ⅓ of the time

Exponentials have ~250× less throughput than matrix math (3.9 vs. 989 TFLOPS)



EACH BLOCK OF ATTENTION

1 · Load K, V block · TMA copy engine

2 · Scores: S = QK^T · tensor cores

3 · P = softmax(S) · CUDA cores

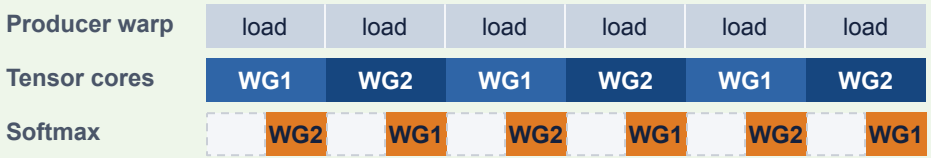
4 · Output: O += PV · tensor cores

Warp: 32 threads · Warpgroup: 4 warps

Barrier: a warp waits for another's signal

Overlapped: loads run ahead, warpgroups take turns

Full FA3: 661 TFLOPS. Without producer warps: 570. Without softmax–matmul overlap: 582.



Head dim 128 · Blue: matmul · Orange: softmax · Gray: loads · Dashed: idle · Source: Shah et al., FlashAttention-3 (2024)

Tuning kernels can significantly increase GPU throughput. Why don't we tune all kernels?

Hand-Tuning Every Kernel Doesn’t Scale

What we just saw is written by hand, for every operation, on every GPU.

1 · SLOW TO WRITE

Thousands of lines, weeks of work

Getting the overlap right takes specialist skill in data movement and synchronization.

≈8,500 lines

in one TensorRT-LLM attention decode kernel; 2–8 weeks of expert work per kernel, per platform

2 · TIED TO ONE GPU

Redone for each hardware generation

FlashAttention-2 was hand-tuned for the A100. On the newer H100, it couldn’t use the new features.

70% → 35%

FlashAttention-2’s share of peak, A100 vs. H100: the gap FlashAttention-3 was written to close

3 · HIGH DEMAND

New operations, new chips, new kernels

New model designs bring new operations, and each one is needed on every chip.

1,500+

ads-ranking models in production at Meta, across NVIDIA, AMD and its own chips

Sources: Shah et al., FlashAttention-3 (2024); Liao et al., KernelEvolve (Meta, 2025), §1

Hand-tuning can’t keep up. Can agents write these kernels instead?

Agents Write Fast Kernels, but Can’t Yet Be Trusted

AGENTS HAVE SHOWN HIGH POTENTIAL

Starting point, early 2025 · KernelBench
Top models matched PyTorch on fewer than 20% of tasks

Early 2025 · METR
o3-mini-high averaged 1.81× over PyTorch; speedups doubled in six months

Late 2025 · KernelEvolve (Meta)
Correct on all 250 KernelBench tasks; weeks of work cut to hours

Early 2026 · CUDA Agent
Faster than torch.compile on 92–100% of tasks

BUT THEY LEARN TO BEAT THE CHECK

Sakana AI CUDA Engineer, Feb 2025
Claimed up to 100× speedups; its kernels exploited flaws in the evaluation and skipped the accuracy check. Claims walked back.

KernelBench audit, 2025
Kernels that just called PyTorch, reused memory already holding the answer, or ran on a stream the timer didn’t measure

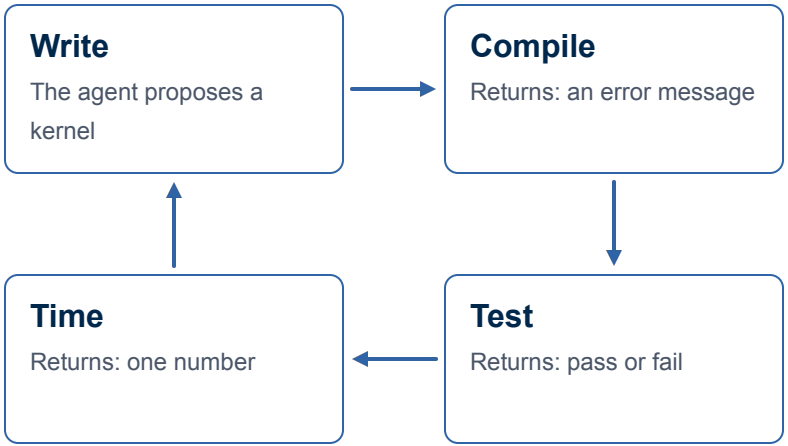
Kevin-32B training, 2025
Copied the reference, or fell back to PyTorch whenever its CUDA failed

Sources: Ouyang et al. 2025; METR 2025; Liao et al. 2025; Dai et al. 2026; TechCrunch 2025; KernelBench v0.1 blog; Cognition 2025

Some speedups are real and some are hacks, and a passing test can’t tell them apart.

Agents Learn Little From Each Attempt

Even when results are honest, each attempt teaches little. Every agent runs this loop:



WHAT THE AGENT HEARS

“Failed.” “Hung.” “Passed, 0.94 ms.”

Nothing says which decision caused it.

WHAT IT NEEDS TO HEAR

“Warpgroup 2 reads stage 2 before the producer warp’s TMA load has filled it.”

A specific decision to fix, found before spending GPU time

Stage: one of several buffers the producer warp fills in rotation

When a kernel fails or runs slow, the agent can’t tell which decision to fix.

An Agent-Facing Kernel Interface Needs Four Things

1

Expose the decisions that set speed

Warp roles, pipeline stages and data placement, written explicitly

Because speed comes from the schedule

2

Grow with new hardware and workloads

Add new features and new checks as gaps appear

Because GPUs change every 1–2 years

3

Verify results independently

Checks outside the agent's control: a trusted reference, many inputs, robust timing

Because a pass isn't proof the kernel is real

4

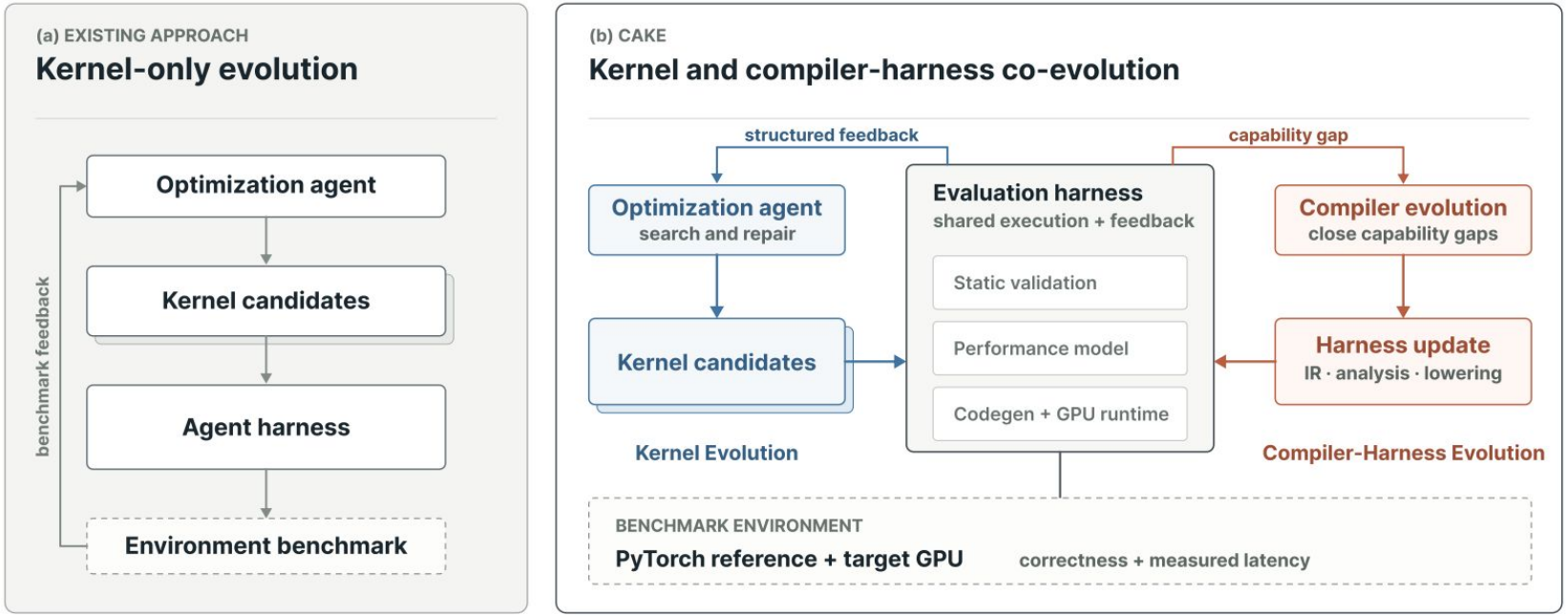
Explain failures and slowdowns at their source

The decision at fault, the slowest stage, and where roles wait on each other

Because pass/fail and one time say little

Next: how CAKE builds a compiler that meets these requirements.

CAKE Co-Evolves GPU Kernels and the Compiler

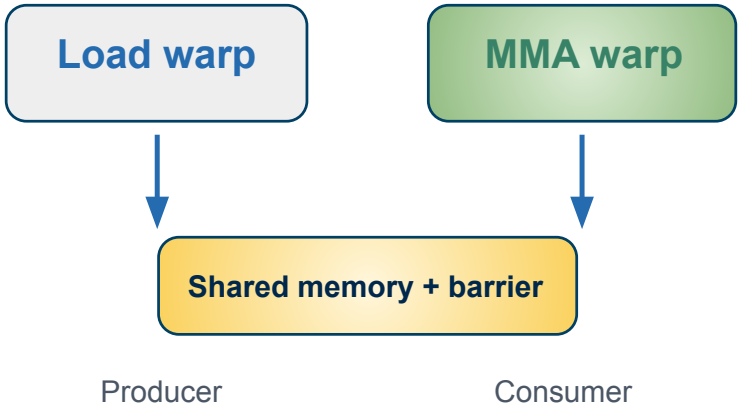


Cake IR Makes Hardware Scheduling Explicit and Analyzable

```
load = lm.role(warps=[0])
mma  = lm.role(warps=[1])

pipe = lm.pipeline(stages=3)

q_full = lm.barrier(
   count=3,
   prod=[load], cons=[mma],
   init_count=1, pipeline=pipe)
```



The compiler can identify which role produces data and which must wait.

Explicit Roles and Barriers Enable Static Verification



Blocking checks return localized reasons. Performance reports guide search, not final acceptance.

The agent gets a repair target, not just a crash or latency number.

Structured Compiler Feedback Guides Kernel Optimization

Path A · Missing capability

Production kernels + hardware docs

↓

New IR primitives + lowering

Path B · Repeated failure

Sanitizers + errors + profiler evidence

↓

New verifier checks / calibration

↓

Corpus regression tests + human merge gates

A new workload can improve the compiler for subsequent workloads.

Recurring Failures Drive Compiler Evolution

INNER LOOP — KERNEL EVOLUTION

Generate candidate → Verify + rank → GPU test → Repair



OUTER LOOP — COMPILER EVOLUTION

Gather failures + missing patterns → Update harness → Corpus tests

Kernel evidence guides the next candidate, and can improve the environment.

Experimental Setup: A Controlled Flash-KMeans Comparision

CAKE IR

Agent authors typed Cake IR

Verifier + cost model + lowering

Direct CUDA/PTX

Agent authors CUDA C++ / inline PTX

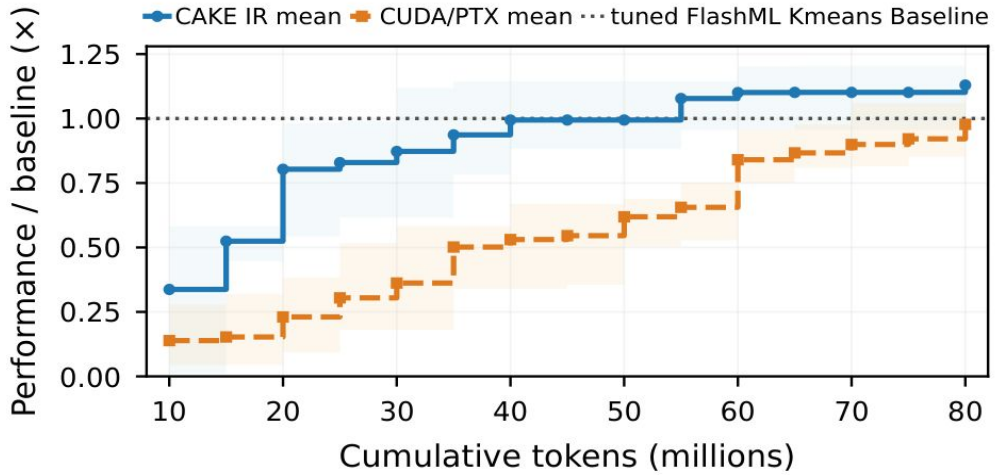
Direct low-level implementation

MATCHED EXPERIMENTAL CONDITIONS

Model + scaffold	GPT-5.6-sol · same agent setup	**Runs + budget**	3 clean starts per arm · 80M tokens
GPU	NVIDIA B200	**Correctness**	Same oracle, tolerances, and benchmark
Workload	Flash-KMeans assign · single fixed shape	**Reference access**	Low-level target implementation hidden

Same agent, workload, hardware, and budget; only the programming environment changes.

Cake IR Helps Agents Discover Faster Kernels



1.144×
Cake IR median best

0.928×
Direct CUDA/PTX

Matched protocol

Same agent · B200 · 3 runs per arm · implementation hidden

Representation + feedback matter even when the coding model is held fixed.

CAKE Extends Beyond Single-Shape Kernel Optimization

FRONTIER KERNEL SYNTHESIS

2.05×

Kimi Delta Attention prefill

Implementation hidden · six B200 shapes · end-to-end serving checked

DISPATCHER-BACKED KERNEL FAMILIES

434 shapes

KNN build · KNN search · KMeans

1.418× · 2.116× · 1.803× geometric-mean speedups

Frontier synthesis tests discovery; dispatcher results test generalization.

CAKE Improves Agent-Driven Optimization, but Challenges Remain

WHAT THE PAPER SHOWS

- ✓ **Hardware-explicit, analyzable schedules**
- ✓ **Localized correctness + performance evidence**
- ✓ **Reusable compiler capabilities**

WHAT REMAINS OPEN

- Performance evidence mainly on B200
- Cost model calibrated only for B200 / H100
- Incomplete analysis; human merge gates

Main idea: evolve the compiler environment, not only the GPU kernel.

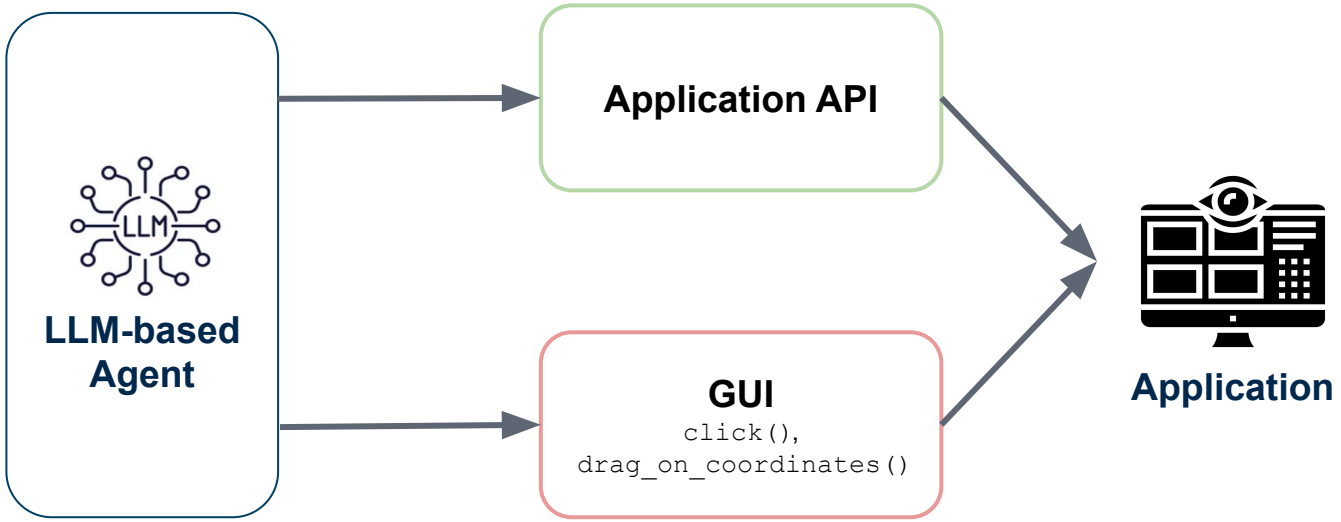
MICHIGAN ENGINEERING
UNIVERSITY OF MICHIGAN

From Imperative to Declarative: Towards LLM-friendlyOS Interfaces for Boosted Computer-Use Agents

Yuan Wang, Mingyu Li, Haibo Chen

How Do Computer-Use Agents (CUAs) Interface with the OS?

The application interface restricts **CUAs**, unlike coding agents that navigate through purely natural language.



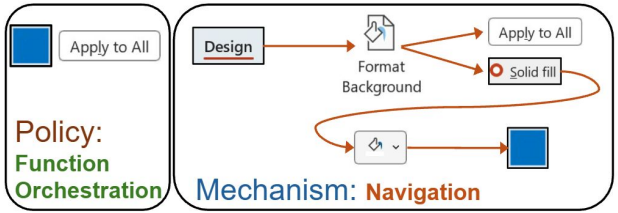
The **imperative** nature of LLM-GUI interaction ultimately increases inference costs and risk of failure.

Declarative Model Interface (DMI): Offload Mechanism

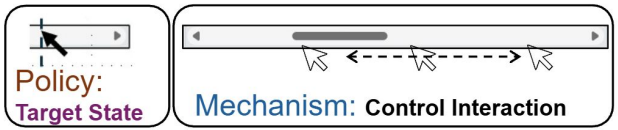
GUIs Entangle Policy and Mechanism

Policy: Semantic intent

Mechanism: How to accomplish policy



(a) Task 1: make the background blue on all slides.



(b) Task 2: show the area close to the end.

Figure 2. Policy-Mechanism coupling in GUI use.

DMI executes interaction logic independently.

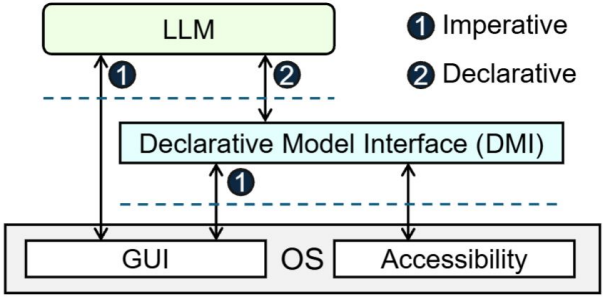


Figure 1. Overview of the DMI abstraction layer.

`access`	`state`	`observation`
Navigate to element and perform an action	Set an element's state.	Return an element's state as structured info

Fewer LLM decisions → ↓ token cost ↓ latency ↑ task success

MICHIGAN ENGINEERING
UNIVERSITY OF MICHIGAN

Why do GUIs Create Obstacles for LLMs?

GUIs are **human-centric OS designs**, so they are **mismatched** with the strengths of LLMs.

Mismatch #1: Procedural Navigation

GUIs expose functionality step-by-step to reduce cognitive load on humans.

LLMs reliably handle cognitive load (**context windows**) but struggle with **vision**.

Without a global view, long navigation sequences create **fragile action chains**.

Mismatch #2: Iterative Interaction

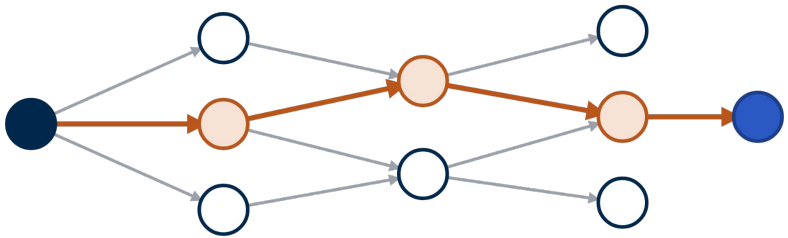
GUI interaction is an iterative "**observe-act**" loop.

- **Text selection:** Repeated cursor movements from the start to end.

	USERS	LLMS
Loop Cost	3-5 mouse adjustments per sec	10-120+ seconds per round-trip
Effort	Low effort, accurate perception	Poor visual acuity

Insight: Navigation is Deterministic

Control: UI control element exposed by an **accessibility API** (e.g. button, text box)



Nodes: Controls
Edges: Interactions

Navigation becomes a **deterministic problem** because an app's control transitions are **fixed**.

Accessibility APIs Make Interactions Finite

Windows UI Automation (UIA) labels every control with:

Type (what it is, 41 total)

`Button, ListItem, Edit` …

Pattern (what it can do, 34 total)

`ScrollPattern, TextPattern, SelectionPattern` …

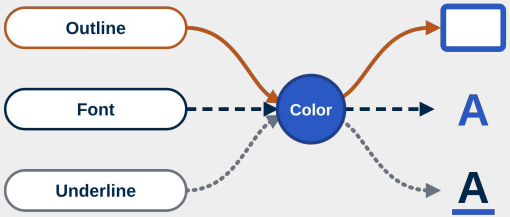
Unique Paths → Declare Only the Target

Disambiguating the graph gives each control one path. Given just the target, DMI can navigate deterministically.

Design Challenges

1. Path Ambiguity

A control's meaning depends on the path used to reach it.



2. Limited Context Window

Full navigation topology is too costly in prompts.

5,000+
controls in Microsoft Office

3. Dealing with Instability

Navigation/Interaction
to/with controls may fail.

Long-Horizon Planning:
Failures cascade

DMI Overview: Offline Graph Modeling, Online Execution

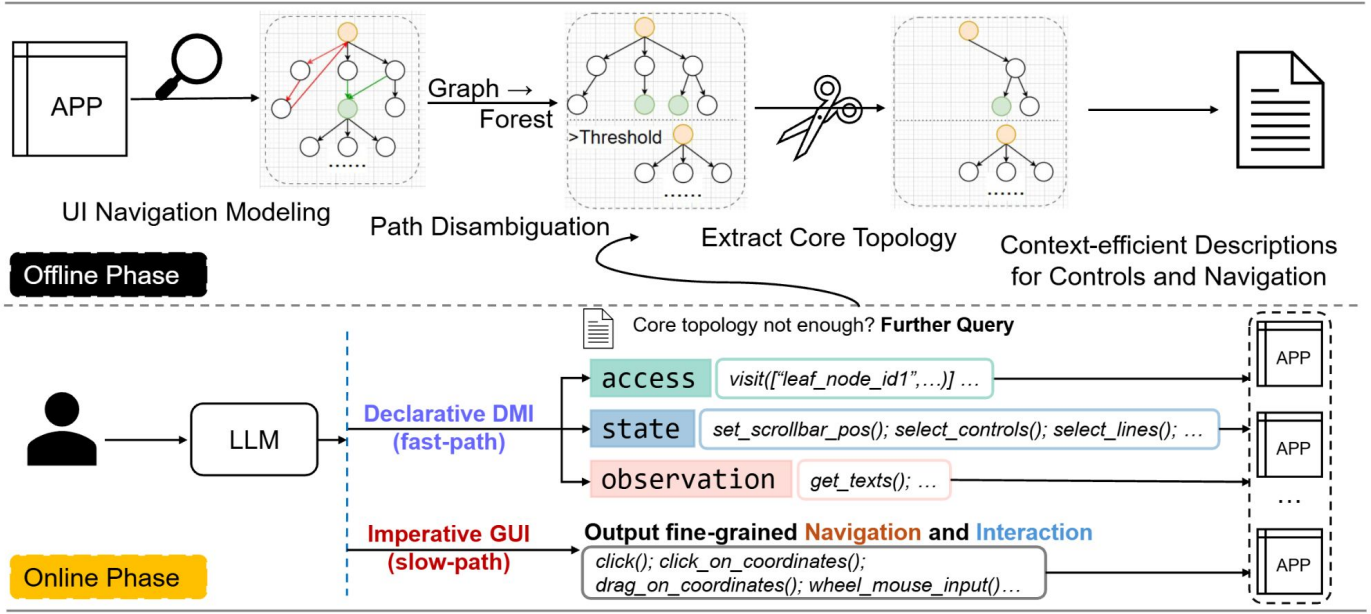


Figure 3. DMI Workflow: Offline Modeling and Online Execution

Path Disambiguation: Graph → DAG → Forest

Goal: Achieve unambiguous control access by control ID. **How:** Disambiguate so every control has a unique access path.

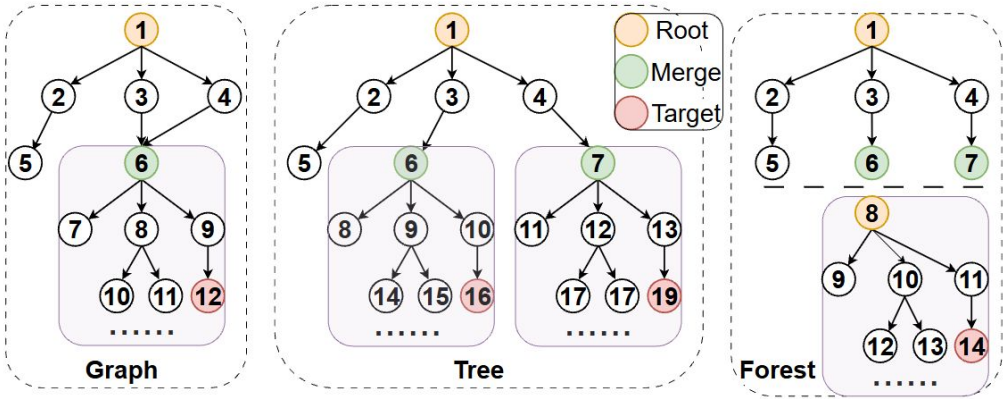


Figure 4, Navigation Topology.

Graph: many paths

Cycles and **merge nodes** result in control IDs being accessible by multiple paths.

Tree: decycle + duplicate all

Cloning **merge nodes** give unique IDs, but the node count **explodes**, often exceeding the context window,.

Forest: externalize selectively

Clone cheap merges, share costly merge nodes as subtrees. Ensures **linear growth**.

The LLM only only states **where to go**, and DMI resolves the path.

Context-Efficient Description: Representing the Topology

A **control** is defined by its properties and its navigation path.

Compressed Hierarchy: References

A control = **properties (UIA)** + **hierarchical path**.

Hierarchical Path: Reference ID Disambiguates path

› ref: <id> → Colors › **Blue**

Shared subtrees are represented in a **shared subtree entry map** that includes reference IDs.

Query on Demand

Core topology: Limit depth, omit large lists (e.g. fonts), manually identified nodes

≈1K	≈1K
Word	PowerPoint

From ~4K+ originally.

Query on-demand: Specific branches or full topology

```
{"further_query": ["<node_id> OR <-1>"]}
```

Navigation: The `visit` Interface

Illustrative Commands

Access: Navigate to control and perform an action.

```
{"id": "16"}
```
access a control

```
{"id": "6", "entry_ref_id": ["5"]}
```
access inside a shared subtree

```
{"id": "21", "text": "Q3"}
```
access + type text

```
{"shortcut_key": "ENTER"}
```
e.g., commit an edit

```
visit([{"id": "13"}, {"id": "16"}])
```
chain several commands in one call

Balance Accuracy and Efficiency

- **Efficiency:** Combine navigation with action. Chain commands.
- **Accuracy:** Prohibit visit and state/observation in the same call to force observance
  - **Exception:** Text input is allowed to chain

Handle UI Instability

- **Fuzzy matching:** Match controls by type, ancestors and name
- **Path recovery:** No match → close the window (OK > Close > Cancel)
- **Error Feedback + retry:** report states like “disabled”; retry slow controls

Handle LLM Instruction-Following Errors

- LLMs might emit navigation steps; DMI drops non-leaf IDs and runs only the leaf targets

Interaction: State and Observation Interfaces

DMI uses UIA control patterns to provide pre-wrapped, high-value patterns (extensible).

State
Input: Desired end-state for a control
Result: Transition control from current state to target end-state.

Observation
Input: Information request for a control
Result: Structured data representing the information

Interface	UIA pattern	Definition	Interaction type
set_scrollbar_pos	Scroll	Set a scrollbar to x%	State
select_controls	Selection	Select one or several controls	State
get_texts	Text, Value	Return a control's text	Observation

Passive (Pre-emptive) vs. Active versions of `get_texts()`

Before each LLM call, invoke `get_texts()` on all `DataItem` controls and forward a truncated result into the prompt.

Test Setup

Applications

Word, Excel and PowerPoint (Microsoft 365) on Windows. Each has **5,000+ controls**, nesting deeper than 10 levels, with cycles and many merge nodes.

Benchmark

OSWorld-W: all **27 single-app tasks** for the three apps. Multi-app tasks excluded, since they need the OS's own controls modeled.

Baseline: UFO2-as

Microsoft's **UFO-2, GUI-only, plus action sequences** (several actions per turn, **visible controls only**). Office COM APIs excluded as not general.

Method: UFO2-as + DMI

Prompted to prefer DMI, falling back to GUI. 30-step cap, 3 runs per task. **GPT-5 (medium, minimal)** and **GPT-5-mini**; no fine-tuning.

Offline Modeling: UI Navigation Modeling Cost

< 3 h

Automated GUI ripping per application

≈ 1.5

person-days of manual setup per app (blocklist, object contexts)

App	Raw graph	Core	Tokens	Shape
Excel	> 4K	≈ 2K	≈ 30K	forest
Word	> 4K	≈ 1K	≈ 15K	forest
PowerPoint	> 4K	≈ 1K	≈ 15K	single tree

Counts are controls; tokens are the core topology added prompt size. No nested references among shared subtrees.

The model is **version-specific** but reusable across machines running the same application build.

DMI Improves Success Rate and Reduces Interaction Steps

SR = Average Success Rate

Interface	Knowledge	Model	Reasoning	SR	Steps	Time(s)
GUI-only	/	GPT-5	Medium	44.4%	8.16	392
GUI-only	Nav.forest	GPT-5	Medium	42.0%	8.41	353
GUI+DMI	**Nav.forest**	**GPT-5**	**Medium**	**74.1%**	**4.61**	**239**
GUI-only	/	GPT-5	Minimal	23.5%	8.42	251
GUI+DMI	**Nav.forest**	**GPT-5**	**Minimal**	**40.7%**	**5.52**	**140**
GUI-only	/	5-mini	Medium	17.3%	7.14	171
GUI-only	Nav.forest	5-mini	Medium	23.5%	6.32	**150**
GUI+DMI	**Nav.forest**	**5-mini**	**Medium**	**43.2%**	**4.43**	167

1.67×
task success rate

−43.5%
LLM steps per task

Note: All metrics are calculated for successfully completed tasks only.

Over 61% of successful DMI runs finish in 4 steps

Step 1 · HostAgent

Splits the task by app and opens the target app

Step 2 · AppAgent

Executes the subtask: one DMI call does it all

Step 3 · AppAgent

Verifies the result and decides on handoff

Step 4 · HostAgent

Verifies overall task completion

Baseline action sequences

Can batch actions, but only on controls already visible, so it must stop, observe and re-plan after every menu opens.

Declarative DMI

Plans globally over the forest, including hidden controls, so the core intent fits in a single LLM call.

The Cost of DMI's Performance Improvements

Token cost

+30K Excel

+15K Word

+15K PowerPoint

- About 15 tokens per control (o200k_base encoding)
- Over 80% of the extra context is the navigation forest; the rest is the DMI prompt and DataItem text
- Fewer rounds, so **total tokens per task are lower** than the baseline (core setting)

Per-step latency

LARGE MODEL

Per-call latency similar to the baseline, with or without reasoning.

SMALL MODEL

Each call is slower, but total task time still drops because there are fewer calls.

Ablation Study: The Declarative Interface Drives the Gains

The baseline is given the same navigation forest but without access to DMI

Configuration	GPT-5 SR	GPT-5 steps	5-mini SR	5-mini steps
GUI-only	44.4%	7.94	17.3%	7.02
GUI-only + forest in prompt	42.0%	8.58	23.5%	6.26
GUI + DMI	**74.1%**	**4.60**	**43.2%**	**4.52**

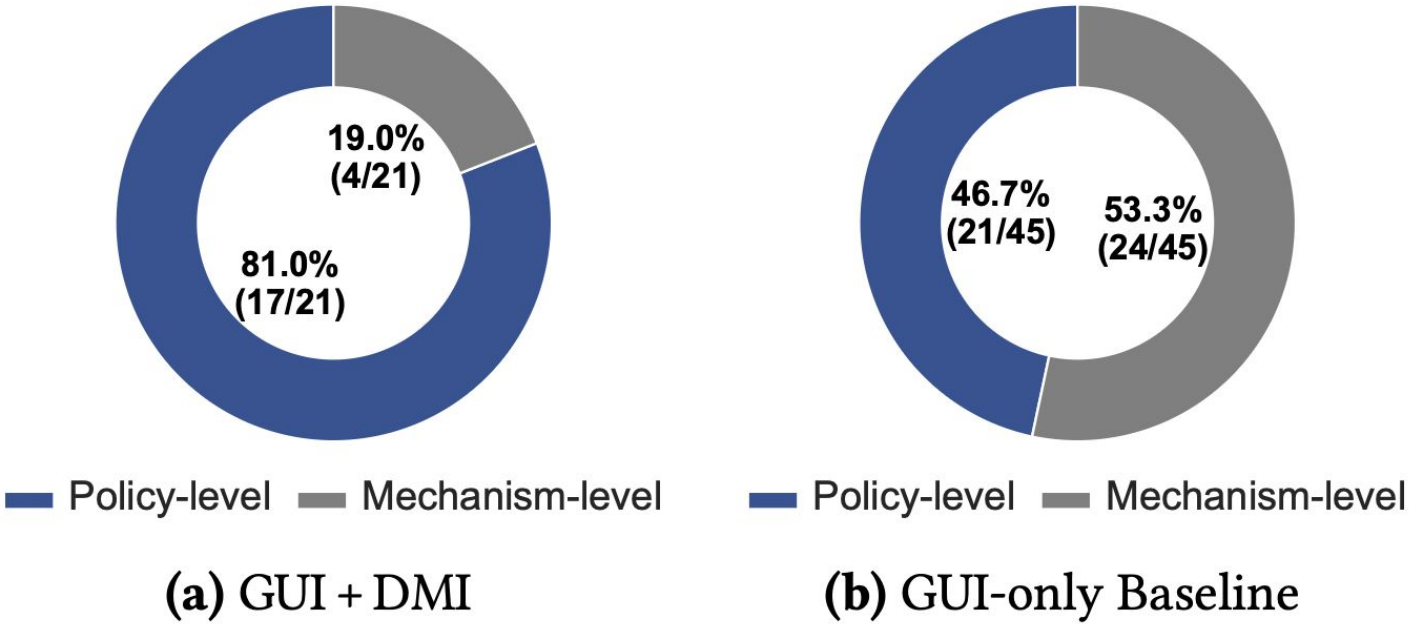
GPT-5: Knowledge alone does nothing

The forest leaves success flat (42.0% vs 44.4%) and steps slightly worse.

GPT-5-mini: It helps a little

+6.2 points from the forest, but +25.9 points from the full DMI interface.

Failure Analysis: DMI Shifts Errors from Mechanism to Policy



Acknowledged Limitations

1. **Not every interaction is declarative**
   Freehand drawing and pixel-precise moves still need the slow GUI path.

2. **Modeling cost per app and version**
   One navigation graph per app build (≈1.5 person-days of setup); only Windows UIA is built.

3. **Apps without accessibility support**
   Legacy XP-era apps, games and graphics software often expose no UIA tree.

4. **Incomplete navigation topology**
   Ripping misses conditional UI changes; the GUI fallback then relies on weak vision.

5. **Dynamic UIs such as the web**
   Changing web DOMs resist pre-modeled access; state and observation still carry over.

MICHIGAN ENGINEERING
UNIVERSITY OF MICHIGAN

Questions for discussion

1. 27 tasks × 3 runs is a small benchmark. How stable would the gains be on a larger task set?
2. Models are tied to an app build. Who rebuilds the forest when Office updates monthly?
3. DMI depends heavily on accessibility metadata. How robust is the approach when controls have ambiguous names, missing identifiers, or poor semantic descriptions?
4. Should applications ship their own navigation forest, like an accessibility manifest for agents?

MICHIGAN ENGINEERING
UNIVERSITY OF MICHIGAN

Conclusion: OS interfaces should evolve for LLM agents

Declarative over imperative. The LLM states the goal and DMI works out the navigation and interaction needed to reach it.

Fast path, slow path. DMI handles what it can declare by default, and the GUI remains available for coverage and recovery.

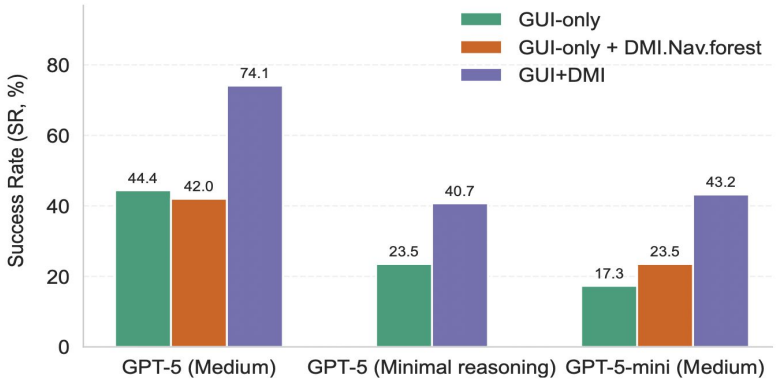
1.67×
task success rate

−43.5%
LLM steps per task

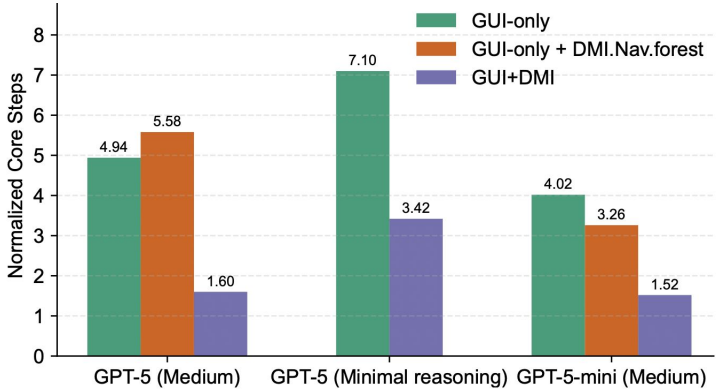
61%+
done in one core LLM call

81%
of failures are policy-level

DMI Improves Success Rate and Reduces Interaction Steps



(a) Success rate.



(b) Normalized Core steps.

Previously, Optimized AI Systems From Cluster to CPU

BLOCK 1

Scaling and orchestrating systems

Tessera · Llama 3 training · HybridFlow · RollArt · Cornfigurator · DiFlow · Pie · Murakkab

BLOCK 2

Making agent workloads efficient

Cost of Dynamic Reasoning · CPU-Centric Agentic AI · Continuum · Strata · Mooncake · SPEX · Speculative Actions

BLOCK 3

Inside the GPU

MPK

TODAY

Interfaces designed for AI agents as the user

DMI (top of the stack) · CAKE (bottom of the stack)

Today's question: what changes when the user of a system is an AI agent?

Today, We Rebuild an Interface for AI Agents

	BUILT FOR PEOPLE	REBUILT FOR AGENTS
DMI Apps and the operating system	**Menus, one click at a time** People handle this easily: sharp eyes, quick looks, no need to remember every menu.	**State the goal - DMI does the clicking** Agents misread screens and every step is a slow model call, but they can hold an app's whole menu map at once.
CAKE The compiler and the GPU	**Raw GPU code and error messages** Experts fill the gaps with knowledge of the workload, the hardware, and years of experience.	**State the plan - the compiler checks it** Agents can try many kernels quickly, but need checks and specific feedback to keep improving.

Both papers redesign the interface around what agents are good and bad at relative to humans

MICHIGAN ENGINEERING
UNIVERSITY OF MICHIGAN

CAKE: Compiler-Agent Co-Design for Frontier Kernel Evolution

(Authors)

What is a GPU Kernel?

~ The Plan for a Banquet Kitchen: Same Meal for 100+ Guests

Walk-in fridge
GPU main memory
Holds everything, but takes longer to reach

→

Counter slots
On-chip memory
Small, but right at the station

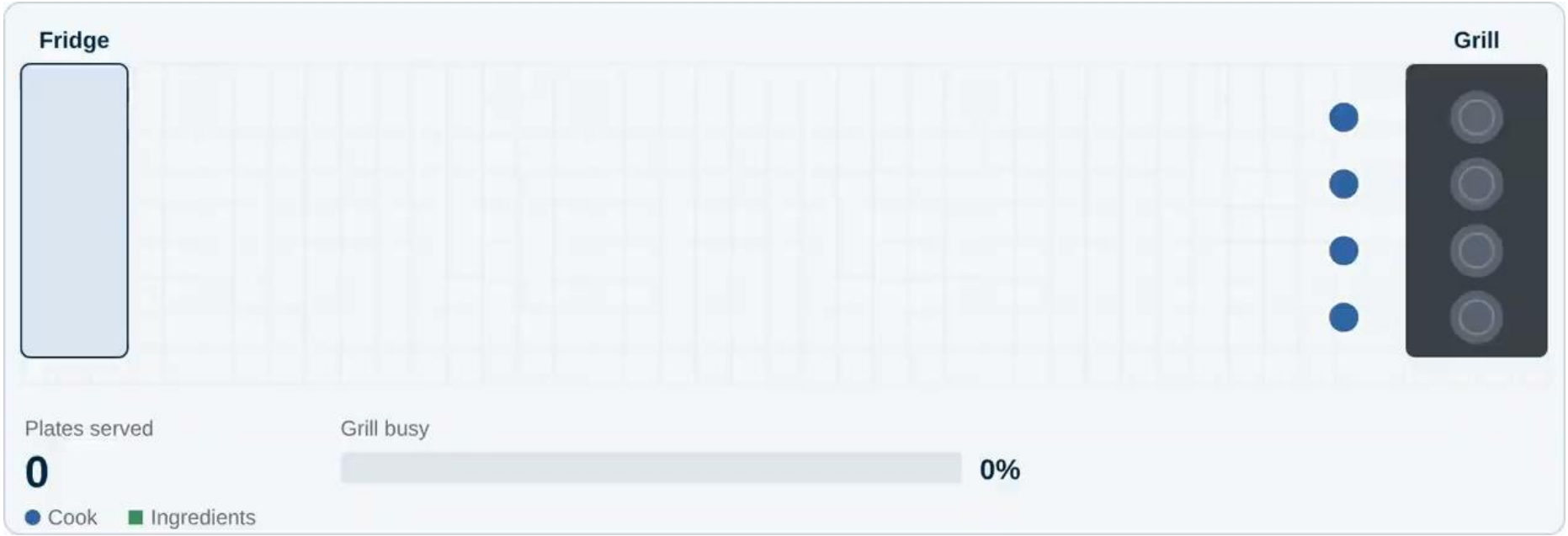
→

Grill
Tensor cores
Matrix math, extremely fast: the most valuable equipment

Cooks
Warps
Groups of 32 threads that run the same steps together

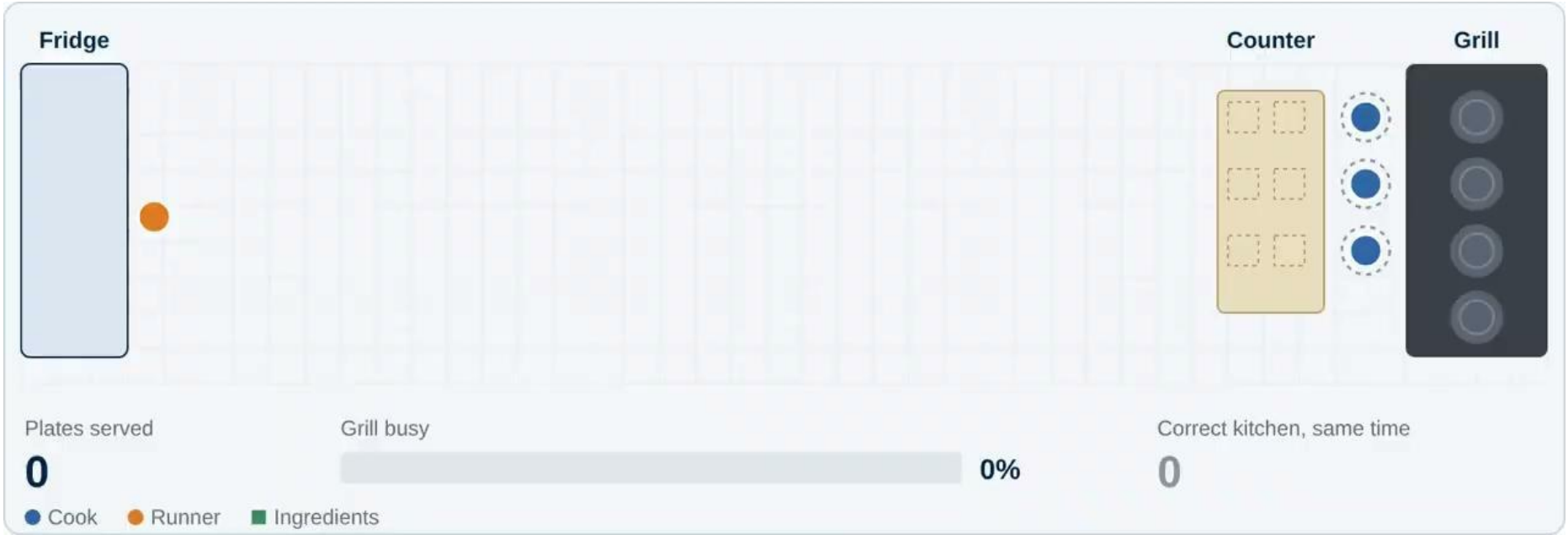
A kernel is the plan for this kitchen to serve all guests correctly as fast as possible

A simple plan might have each cook prepare one dish each in parallel



Animation created using Claude Code

An expert plan would have one cook bring ingredients to the others to keep the grills on



Animation created using Claude Code

Expert Kernels Keep the GPU Cores Busy

Correct kernel: every cook does every step

Cook 1	Fetch	Cook	Fetch	Cook	Fetch	Cook
Cook 2	Fetch	Cook	Fetch	Cook	Fetch	Cook
Grill	idle	busy	idle	busy	idle	busy

Expert kernel: runners fetch while cooks cook

Runner	Fetch 1	Fetch 2	Fetch 3	Fetch 4	Fetch 5	Fetch 6
Cook	wait	Cook 1	Cook 2	Cook 3	Cook 4	Cook 5
Grill	idle	busy the whole time				

SAME MATH, SAME H100

35%

of peak: FlashAttention-2

75%

of peak: FlashAttention-3, which overlaps data movement and math

Source: Shah et al., FlashAttention-3 (2024)

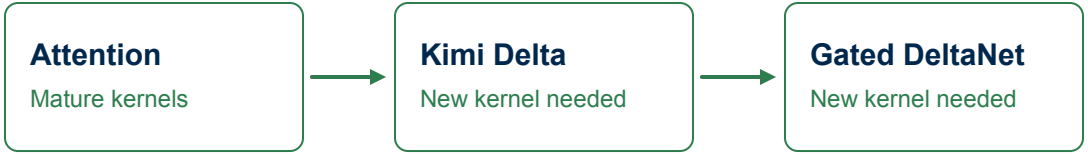
The speedup comes from this overlap.

Expert Kernels Are Rare and Short-Lived

EVERY GPU GENERATION



EVERY NEW MODEL DESIGN



AND EACH ONE IS LARGE

≈2,400

lines of GPU code in FlashAttention-4's forward kernel

≈8,500

lines in a TensorRT-LLM attention decode kernel

Reference implementations, CAKE paper Table 4

Each new GPU or model restarts the work, and it takes rare expertise.

Can Agents Can Write Expert Kernels? – Yes, but Slowly and Unreliably

VS PYTORCH (~ Correct Kernel):

Early 2025 · KernelBench

Top models matched PyTorch on fewer than 20% of tasks

Late 2025 · KernelEvolve (Meta)

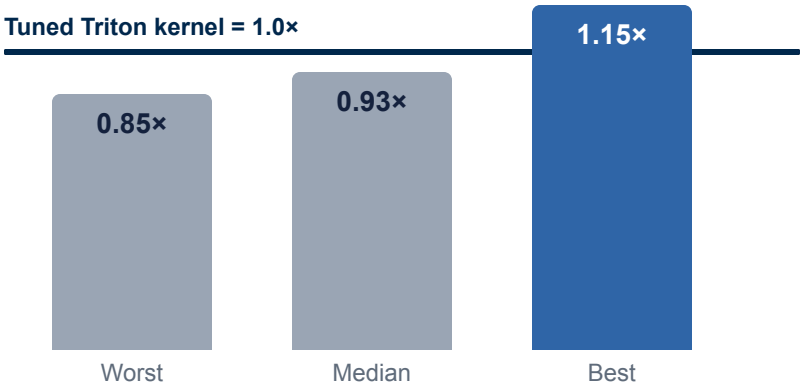
Correct on all 250 benchmark tasks; weeks of work cut to hours

Early 2026 · CUDA Agent

Faster than torch.compile on 92–100% of tasks

REACHING EXPERT LEVEL

Same agent writing raw CUDA, three runs

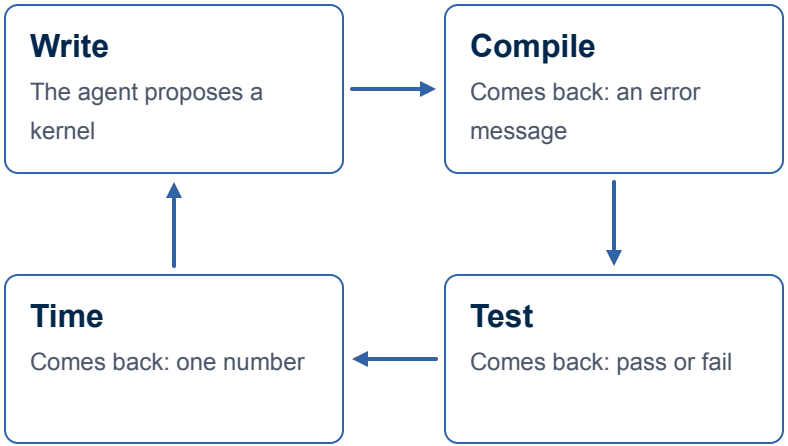


None converged; 3.6–4.3 hours of agent time each

Agents can reach expert speed, but it takes hours and varies from run to run.

Reason 1: Agents Learn Little From Each Failure

Every kernel agent runs this loop:



WHAT THE AGENT HEARS

“It failed.” “It passed.” “0.94 ms.”

A head chef told “bad night,” with no reason given

WHAT IT NEEDS TO HEAR

“The runner refills slot 2 before the cook has emptied it.”

In GPU terms: a barrier between two roles is out of step

Each failed try costs GPU time and teaches the agent almost nothing, requiring more attempts and more failures

Reason 2: Kernel Languages Aren’t Built for Agents

More automatic — More control

TILE LANGUAGES · TRITON

Compiler plans automatically

The compiler decides who fetches, who computes, and when

Problem: the compiler chooses the plan, not the agent
→ Lower speedup ceiling

LOW-LEVEL · CuTe

Full control, through formulas

Every data placement must be written as a math formula

Problem: data placement requires a separate error-prone translation
→ Unreliable results

RAW CUDA / PTX

Full control, by hand

Total control, and nothing is checked for you.

Problem: a slow, error-prone search for an agent
→ Slow, unreliable results

Each option gives the agent too little control or too little safety.

Agents and Compilers Evolved Apart

KERNEL AGENTS

Got better at searching

- Smarter proposals, edits and ranking
- Still treat the compiler as a black box
- Hear only an error, pass or fail, and a time

Expert kernels fall in the gap

GPU LANGUAGES

Got better at control

- Designed for human experts
- Hide the plan, or demand layout math
- Don’t grow when a new kernel needs something missing

Expert kernels need fine control plus feedback an agent can act on. No tool offers both.

Agents Need Control, Clear Feedback, and Memory

1

Control without the math

Say who fetches, who cooks, and which calls connect them.

In CAKE: Cake IR

2

Feedback that names the cause

Which call broke and which station is the bottleneck, before service starts.

In CAKE: verifier and cost model

3

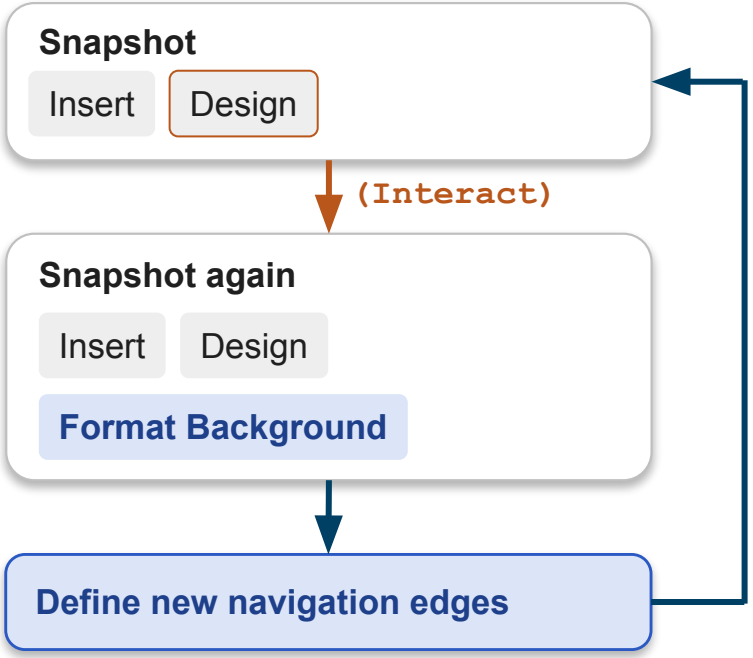
Tools that learn from mistakes

A mistake that keeps happening becomes a permanent checklist rule.

In CAKE: compiler evolution

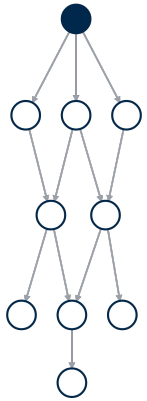
Next: how CAKE builds the compiler around these three needs.

GUI Ripping for Graph Construction



GUI Ripping: Automated exploration of the GUI to model it.

Result: UI Navigation Graph (UNG)



Manual Work (Trade-off)

- **Dead-Ends:** Access blocklist
- **State-Dependent UI:** Additional Context